

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования

Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №2

«Таблицы»

Выполнил:

студент группы 3824Б1ФИ2

Щербаков Н.А.

Проверил:

Ивлев А.Д.

Нижегород
2026

Содержание

Введение	3
1. Постановка задачи	4
2. Руководство пользователя.....	5
3. Руководство программиста	10
3.1. Описание структуры программы	10
3.2. Описание структур данных	10
3.3. Описание алгоритмов.....	22
4. Результаты экспериментов и тестирования	26
Заключение.....	32
Литература.....	33

Введение

Эффективное хранение и обработка данных — одна из ключевых задач программирования. Выбор структуры данных оказывает прямое влияние на скорость выполнения операций вставки, поиска и удаления элементов. Таблицы, отображающие уникальные ключи в значения, являются фундаментальным инструментом во многих приложениях: от баз данных до систем компьютерной алгебры.

В данной работе рассматриваются три классических способа организации таблиц: неупорядоченная таблица на основе вектора, упорядоченная таблица на сбалансированном АВЛ-дереве и хэш-таблица с методом цепочек для разрешения коллизий. Каждая из них обладает своими преимуществами и компромиссами между сложностью реализации и скоростью доступа. Значениями в таблицах могут выступать данные произвольных типов, в том числе полиномы трёх переменных, реализованные в предыдущей лабораторной работе.

Программа позволяет не только выполнять операции над таблицами, но и фиксировать количество элементарных операций, затраченных каждым типом таблицы. Это даёт возможность проводить сравнительный анализ эффективности структур данных на практике, строить графики и делать выводы о целесообразности их применения в различных ситуациях.

1. Постановка задачи

Целью данной работы является разработка программы на языке C++, реализующей таблицы трёх типов для хранения данных с подсчётом количества произведённых операций и предоставляющей пользователю консольный интерфейс для работы с ними.

Типы таблиц:

1. Неупорядоченная таблица на векторе;
2. Упорядоченная таблица на сбалансированном AVL-дереве;
3. Хэш-таблица с методом цепочек.

Особенности реализации:

1. Работа происходит одновременно со всеми тремя типами таблиц; каждая операция выполняется над всеми таблицами.
2. Ключ – строка, сравнение лексикографическое.
3. Хэш-функция – полиномиальная, разрешение коллизий – метод цепочек.
4. Пользователь может добавлять записи с уникальным ключом, находить (выводить или извлекать) данные по ключу и удалять записи по ключу.
5. В качестве данных могут использоваться полиномы трёх переменных из лабораторной работы №1, а также данные встроенных типов (целые, вещественные, строки, логические, символы).
6. Консольное приложение поддерживает арифметические операции с полиномами и размещение результатов в таблицах.
7. В процессе работы ведётся логирование количества выполненных операций. По завершении работы логи сохраняются в файл с общей статистикой.
8. Для проверки корректности используется Google Test.
9. Реализованы эксперименты для сравнения эффективности вставки, поиска и удаления элементов в таблицах при увеличении объёма данных.

2. Руководство пользователя

Данная программа предназначена для работы с таблицами трёх типов. Пользователь может создавать полиномы, выполнять над ними арифметические операции, сохранять полиномы и результаты операций над ними в таблицы, вставлять свои данные в таблицы с конкретным ключом, запросить данные из таблиц по ключу, удалить данные из таблиц по ключу и сохранить логики после завершения работы программы.

2.1 Запуск программы

Программа представляет собой консольное приложение с меню. После запуска отображается запрос выбора типа данных для таблиц:

```
+-----+TYPE_VALUE+-----+
0 - integer
1 - double
2 - std::string
3 - Polynomial
4 - bool
5 - char
+-----+ENTER_CHOICE+-----+
```

Рис. 1. Выбор типа данных, с которыми будет работать программа

2.2 Меню программы

После выбора типа данных открывается главное меню программы:

```
+-----+MENU+-----+
1 - Creating/Replacing the first polynomial
2 - Creating/Replacing the second polynomial
3 - Add polynomials
4 - Subtract polynomials
5 - Multiply a polynomial by a constant
6 - Multiply polynomials
7 - Insert data into a table
8 - Querying data from a table by key
9 - Delete data from the table by key
10 - Outputting the program menu to the console
0 - Saving logs AND Shutting down the program
+-----+
+-----+ENTER_CHOICE+-----+
```

Рис. 2. Главное меню программы

2.3 Ввод полинома (команды 1 и 2)

При выборе команд 1 или 2 пользователь вводит соответственно первый или второй полином. Полином состоит из набора мономов, которые пользователь вводит постепенно.

Для каждого монома необходимо ввести:

- Коэффициент – вещественное число (не равное нулю).
- Степень монома – целое число от 0 до 999.

Степень монома задаётся в виде трёхзначного числа, где:

- Сотни – степень переменной X.
- Десятки – степень переменной Y.
- Единицы – степень переменной Z.

Пример: Если пользователь вводит Coefficient: 5 и Degree: 231, то это соответствует моному: $5x^2y^3z^1$.

После ввода монома программа предлагает:

- 1 - Continue entering the polynomial (Продолжить ввод полинома со следующего монома)
- 0 - Finish entering the polynomial (Завершить ввод полинома).

После завершения ввода полинома программа автоматически:

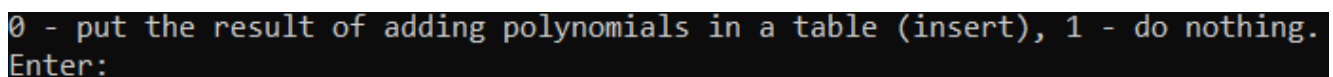
- Удаляет мономы с нулевыми коэффициентами.
- Сортирует мономы по степеням.
- Объединяет мономы с одинаковыми степенями.

2.4 Операции с полиномами (команды 3, 4, 5 и 6)

1. При выборе команды 3 программа выполнит операцию сложения двух полиномов и выведет результат в консоль.

Если какой-либо из полиномов не был введён пользователем, то программа сообщит о том, что соответствующий **полином пуст**.

Если на этапе 2.1 пользователь выбрал в качестве типа данных – Polynomial, то после вывода результата сложения двух полиномов ему предоставляется выбор: сохранить результат операции в таблицы или ничего не делать.



```
0 - put the result of adding polynomials in a table (insert), 1 - do nothing.  
Enter:
```

Рис. 3. Выбор: сохранить или не сохранять результат операции с полиномами в таблицы

Если пользователь выбрал сохранить результат операции в таблицы (введёт 0), то ему предлагается ввести ключ записи. После ввода ключа запись добавляется в таблицы.

2. При выборе команды 4 программа начнёт выполнение операции вычитания.

Пользователю будет предложено:

- 0 - subtract the second from the first polynomial ($p1 - p2$). То есть вычесть второй полином из первого.
- 1 - subtract the first from the second polynomial ($p2 - p1$). То есть вычесть первый полином из второго.

После выбора пользователя результат вычитания будет выведен в консоль.

Если какой-либо из полиномов не был введен пользователем, то программа сообщит о том, что соответствующий **полином пуст**.

Если на этапе 2.1 пользователь выбрал в качестве типа данных – Polynomial, то после вывода результата сложения двух полиномов ему предоставляется выбор: сохранить результат операции в таблицы или ничего не делать (см. рис. 3).

Если пользователь выбрал сохранить результат операции в таблицы (введёт 0), то ему предлагается ввести ключ записи. После ввода ключа запись добавляется в таблицы.

3. При выборе команды 5 программа начнёт выполнение **операции умножения полинома на вещественную константу**.

Пользователю будет предложено:

- 0 - multiply the first(p1) polynomial by a constant. То есть умножить первый полином на константу.
- 1 - multiply the second(p2) polynomial by a constant. То есть умножить второй полином на константу.

После выбора пользователю будет предложено ввести вещественную константу. Результат умножения на константу будет выведен в консоль.

Если соответствующий полином не был введен пользователем, то программа сообщит о том, что **полином пуст**.

Если на этапе 2.1 пользователь выбрал в качестве типа данных – Polynomial, то после вывода результата сложения двух полиномов ему предоставляется выбор: сохранить результат операции в таблицы или ничего не делать (см. рис. 3).

Если пользователь выбрал сохранить результат операции в таблицы (введёт 0), то ему предлагается ввести ключ записи. После ввода ключа запись добавляется в таблицы.

4. При выборе команды 6 программа выполнит **операцию умножения двух полиномов** и выведет результат в консоль.

Если какой-либо из полиномов не был введен пользователем, то программа сообщит о том, что соответствующий **полином пуст**.

Если на этапе 2.1 пользователь выбрал в качестве типа данных – Polynomial, то после вывода результата сложения двух полиномов ему предоставляется выбор: сохранить результат операции в таблицы или ничего не делать (см. рис. 3).

Если пользователь выбрал сохранить результат операции в таблицы (введёт 0), то ему предлагается ввести ключ записи. После ввода ключа запись добавляется в таблицы.

2.5 Вставка данных в таблицы (команда 7)

При выборе команды 7 пользователь вводит ключ для записи, а затем данные, соответствующие выбранному типу на этапе 2.1. Если это полином, то ввод производится аналогично этапу 2.3.

2.6 Поиск данных в таблицах (команда 8)

При выборе команды 8 пользователю предоставляется выбор: выдать или извлечь (выдать и удалить) из таблиц данные по ключу.

```
0 - output the record data by key from the table, 1 - extract record data by key from the table.  
Enter:
```

Рис. 4. Выбор: выдать данные или извлечь данные из таблиц по ключу

После выбора соответствующего действия пользователь вводит ключ, по которому будет производиться поиск записи.

Если запись найдена – программа выведет результат поиска. Если запись не найдена – программа выведет сообщение об отсутствии записи в таблицах.

2.7 Удаление данных из таблиц (команда 9)

При выборе команды 9 пользователь вводит ключ для удаления записи из таблиц.

Если запись с данным ключом существует, то она удаляется из таблиц. Если запись с данным ключом не существует, то программа ничего не делает. В обоих случаях программа выведет сообщение: The removal is completed.

2.8 Вывод меню программы (команда 10)

При выборе команды 10 программы продублирует (выведет) своё меню в консоль.

2.9 Завершение работы программы и логирование (команда 0)

При выборе команды 0 пользователь вводит имя файла, в который программа поместит логи операций и общую статистику.

После ввода имени файла пользователем программа сообщит о сохранении логов и завершится.

2.10 Обработка ошибок

Программа учитывает некорректный ввод пользователя и предоставляет ему возможность повторить соответствующий ввод без завершения программы:

- При некорректном вводе программа выведет: «INVALID INPUT. ‘повтор требований ввода от пользователя’».
- При умножении двух полиномов и выходе за пределы по степеням программа выведет: «The degrees of the obtained variables are greater than 9. The maximum degree of a polynomial is 999.».
- Программа не позволяет выполнить соответствующие операции, когда таблицы пусты.
- Программа не позволяет сохранить файл с логами, если пользователь не ввёл имя файла (то есть если ввёл пустую строку).

2.11 Пример использования программы на целочисленных данных (integer):


```

+-----+ENTER_CHOICE+-----+
7
Enter the key for this record:
first
Enter the integer value for this record:
102943
The insertion is completed.
+-----+ENTER_CHOICE+-----+
7
Enter the key for this record:
second
Enter the integer value for this record:
-3943
The insertion is completed.

```

Рис. 5. Вставка двух записей в таблицы

```

+-----+ENTER_CHOICE+-----+
8
0 - output the record data by key from the table, 1 - extract record data by key from the table.
Enter:
0
Enter the key for this record:
first
The search is completed: 102943
+-----+ENTER_CHOICE+-----+
8
0 - output the record data by key from the table, 1 - extract record data by key from the table.
Enter:
1
Enter the key for this record:
first
The search is completed: 102943
The remove is completed.
+-----+ENTER_CHOICE+-----+
8
0 - output the record data by key from the table, 1 - extract record data by key from the table.
Enter:
0
Enter the key for this record:
first
The record was not found.

```

Рис. 6. Поиск по ключу. Вывод данных с ключом first и извлечение данных с ключом first

```

+-----+ENTER_CHOICE+-----+
9
Enter the key for this record:
second
The removal is completed.
+-----+ENTER_CHOICE+-----+
9
The table is empty.

```

Рис. 7. Удаление данных с ключом second

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из двух проектов (Lab2Code и Lab2Tests), построена по модульному принципу и состоит из следующих компонентов:

- **BaseTable.h:** файл, содержащий абстрактный шаблонный класс, определяющий общий интерфейс для всех типов таблиц. Содержит объявление структуры FindResult, используемой как возвращаемый тип операции поиска, и чисто виртуальные методы insert, find, remove. Здесь же определена глобальная константа EPSILON, применяемая для сравнения вещественных чисел с нулём.
- **UnorderedTable.h:** файл, содержащий реализацию неупорядоченной таблицы на основе std::vector.
- **AVLTable.h:** файл, содержащий реализацию упорядоченной таблицы на сбалансированном АВЛ-дереве. Включает полный набор операций балансировки (малые и большие повороты).
- **HashTable.h:** файл, содержащий реализацию хэш-таблицы, использующей метод цепочек для разрешения коллизий. Содержит полиномиальную хэш-функцию и механизм автоматического рехэширования при превышении коэффициента загрузки loadFactor.
- **MainTable.h:** файл, содержащий главный управляющий класс, который агрегирует все три таблицы, и класс для системы логирования. Предоставляет единый интерфейс insertALL, findALL, removeALL, обеспечивающий синхронное выполнение операций над всеми таблицами.
- **Polynom.h:** файл, содержащий объявления и реализации классов Forward list with f head (односвязный список с фиктивной головой) и Polynomial (полином от трёх переменных). Математическая логика работы с полиномами полностью переиспользована из лабораторной работы №1.
- **Console_app.cpp:** файл, содержащий код консольного приложения. Включает функции main(), print_menu(), input_polynom(), а также специализированные функции для ввода значений различных типов и ключа (input_value_int(), input_value_double(), input_value_string(), input_value_bool(), input_value_char(), input_key()), шаблонные функции insertToTable(), findInTable(), removeFromTable(), управляющие взаимодействием с пользователем, и функцию сохранения результата операций с полиномами saveResultOfOperationWithPolynomials().
- **Tests_Tables.cpp:** файл, содержащий набор модульных тестов, написанных с использованием Google Test. Покрывает тестами все классы таблиц, класс MainTable, классы полиномов, а также содержит эксперименты для сравнения производительности работы с разными типами таблиц.

3.2. Описание структур данных

Основными структурами данных являются три класса таблиц (**UnorderedTable**, **AVLTable**, **HashTable**), которые базируются на классе **BaseTable** и объединены в классе **MainTable**, вспомогательные классы

полиномов (**Forward_list_with_f_head**, **Polynomial**), а также класс для логирования **LoggingOperations**. Ключевой глобальной константой является **const double EPSILON = 1e-9**, которая используется для проверки вещественных чисел на близость к нулю.

1) Структуры *Monom* и *Node*; Классы *Forward list with f head* и *Polynomial*:

Данные структуры данных были подробно описаны в отчёте для лабораторной работы №1 «Арифметические операции с полиномами».

2) BaseTable< TKey, TValue>:

Абстрактный шаблонный класс, задающий общий интерфейс для всех типов таблиц. Определяет базовые операции: вставка, поиск, удаление, очистка, получение размера и имени таблицы. Также содержит вспомогательную структуру для возврата результата поиска с учётом количества выполненных операций.

Внутренняя структура FindResult:

- Поле **TValue *value** – указатель на найденное значение (nullptr, если элемент не найден).
- Поле **size_t operationsCount** – количество элементарных операций, затраченных на поиск.
- Метод **FindResult() : value(nullptr), operationsCount(0) {}**

Функционал: Конструктор по умолчанию. Инициализирует указатель значением nullptr и счётчик операций нулём.

Методы класса:

➤ **virtual size_t insert(const TKey& key, const TValue& value) = 0;**

Функционал: Чисто виртуальный метод. Вставляет элемент с заданным ключом и значением в таблицу или обновляет значение при существующем ключе.

Возвращаемое значение: size_t, количество выполненных операций.

Параметры: key – ключ элемента; value – значение элемента.

➤ **virtual FindResult find(const TKey& key) = 0;**

Функционал: Чисто виртуальный метод. Выполняет поиск элемента по ключу.

Возвращаемое значение: FindResult, указатель на найденное значение и количество операций.

Параметры: key – ключ для поиска.

➤ **virtual size_t remove(const TKey& key) = 0;**

Функционал: Чисто виртуальный метод. Удаляет элемент с заданным ключом из таблицы.

Возвращаемое значение: size_t, количество выполненных операций.

Параметры: key – ключ удаляемого элемента.

➤ **virtual std::string getTableName() const = 0;**

Функционал: Чисто виртуальный метод. Возвращает имя типа таблицы.

Возвращаемое значение: std::string, строка с названием таблицы.

➤ **virtual void clear() = 0;**

Функционал: Чисто виртуальный метод. Очищает таблицу, удаляя все элементы.

➤ **virtual size_t size() const = 0;**

Функционал: Чисто виртуальный метод. Возвращает текущее количество элементов в таблице.

Возвращаемое значение: size_t, количество элементов в таблице.

3) **UnorderedTable< TKey, TValue>:**

Класс неупорядоченной таблицы, реализованный на основе динамического массива (std::vector). Элементы хранятся в произвольном порядке. Поиск, вставка и удаление выполняются линейным перебором.

Внутренняя структура TTableRecord:

- Поле **TKey** key – ключ записи.
- Поле **TValue** value – значение записи.

Поля класса:

- **std::vector<TTableRecord>** records – динамический массив записей таблицы.

Методы класса:

➤ **std::string getTableName() const override**

Функционал: Возвращает название таблицы.

Возвращаемое значение: std::string, строка "UnorderedTable".

➤ **void clear() override**

Функционал: Удаляет все элементы из таблицы.

Сложность: O(n) – за счёт вызова метода clear() для std::vector.

➤ **size_t size() const override**

Функционал: Возвращает количество элементов в таблице.

Возвращаемое значение: size_t, количество элементов в таблице (размер таблицы).

➤ **TValue& operator[](const size_t index)**

Функционал: Обеспечивает доступ к значению элемента по индексу.

Возвращаемое значение: TValue&, ссылка на значение элемента.

Параметры: index – индекс элемента в массиве.

➤ **size_t insert(const TKey& key, const TValue& value) override**

Функционал: Метод для вставки записи в таблицу. Выполняет линейный поиск ключа. Если ключ найден – обновляет значение. Если ключ не найден – добавляет новую запись в конец массива.

Возвращаемое значение: size_t, количество выполненных операций.

Параметры: key – ключ записи; value – значение записи.

Сложность: O(n) – за счёт линейного прохода по элементам вектора в цикле for, где n – количество записей в таблице (размер таблицы).

➤ **FindResult find(const TKey& key) override**

Функционал: Выполняет линейный поиск элемента по ключу. При нахождении возвращает указатель на значение.

Возвращаемое значение: FindResult, указатель на найденное значение и количество операций.

Параметры: key – ключ поиска.

Сложность: $O(n)$ – за счёт линейного прохода по элементам вектора в цикле for, где n – количество записей в таблице (размер таблицы).

➤ **size_t remove(const TKey& key) override**

Функционал: Метод для удаления записи из таблицы. Выполняет линейный поиск элемента. При нахождении заменяет его последним элементом массива и удаляет последний элемент (уменьшает размер).

Возвращаемое значение: size_t, количество выполненных операций.

Параметры: key – ключ удаляемого элемента.

Сложность: $O(n)$ – так как производится поиск удаляемого элемента по ключу, то есть проход по элементам вектора в цикле for, где n – количество записей в таблицы (размер таблицы).

4) AVLTable< TKey, TValue>:

Класс упорядоченной таблицы, реализованный на основе сбалансированного бинарного дерева поиска (АВЛ-дерева).

Обеспечивает логарифмическую сложность операций вставки, поиска и удаления за счёт автоматической балансировки дерева после каждой модификации. Балансировка осуществляется с помощью малых и больших поворотов.

Внутренняя структура Node:

- Поле **TKey** key – ключ узла.
- Поле **TValue** value – значение узла.
- Поле **Node*** left – указатель на левое поддерево.
- Поле **Node*** right – указатель на правое поддерево.
- Поле **int** height – высота узла (используется для балансировки).
- Метод Node(**const TKey&** key_, **const TValue&** value_) : key(key_), value(value_), left(**nullptr**), right(**nullptr**), height(1) { }

Функционал: Конструктор узла. Создаёт новый узел дерева с заданным ключом и значением, инициализирует указатели на потомков как **nullptr** и устанавливает высоту равной 1 (лист).

Параметры: key_ – ключ; value_ – значение.

Поля класса:

- **Node*** root – указатель на корень дерева.
- **size_t** NodeCount – текущее количество элементов в таблице.

Методы класса:

➤ **int getHeight(Node* node_, size_t& opCount) const**

Функционал: Приватный метод. Возвращает высоту узла (0, если узел равен **nullptr**). Используется при вычислении баланса.

Возвращаемое значение: int, высота узла.

Параметры: node_ – указатель на узел; opCount – счётчик операций.

➤ **int** getBalanceIndicator(**Node*** node_, **size_t&** opCount) **const**

Функционал: Приватный метод. Вычисляет фактор баланса узла (разность высот правого и левого поддеревьев).

Возвращаемое значение: int, значение баланса.

Параметры: node_ – указатель на узел; opCount – счётчик операций.

➤ **void** updatedHeight(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Пересчитывает высоту узла на основе высот его потомков.

Параметры: node_ – указатель на узел; opCount – счётчик операций.

➤ **Node*** smallLeftRotation(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Выполняет малый левый поворот.

Возвращаемое значение: Node*, новый корень поддерева.

Параметры: node_ – указатель на корень поддерева; opCount – счётчик операций.

➤ **Node*** smallRightRotation(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Выполняет малый правый поворот.

Возвращаемое значение: Node*, новый корень поддерева.

Параметры: node_ – указатель на корень поддерева; opCount – счётчик операций.

➤ **Node*** bigLeftRotation(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Выполняет большой левый поворот (двойная операция: сначала правый поворот правого поддерева, затем левый поворот текущего узла).

Возвращаемое значение: Node*, новый корень поддерева.

Параметры: node_ – указатель на корень поддерева; opCount – счётчик операций.

➤ **Node*** bigRightRotation(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Выполняет большой правый поворот (двойная операция: сначала левый поворот левого поддерева, затем правый поворот текущего узла).

Возвращаемое значение: Node*, новый корень поддерева.

Параметры: node_ – указатель на корень поддерева; opCount – счётчик операций.

➤ **Node*** balancingTree(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Выполняет балансировку узла: вычисляет фактор баланса и при необходимости применяет соответствующий поворот.

Возвращаемое значение: Node*, указатель на сбалансированный узел.

Параметры: node_ – указатель на узел; opCount – счётчик операций.

➤ **Node*** insertNode(**Node*** node_, **const TKey&** key, **const TValue&** value, **size_t&** opCount)

Функционал: Приватный метод. Рекурсивно вставляет новый элемент в дерево, затем выполняет балансировку.

Возвращаемое значение: Node*, новый корень поддерева.

Параметры: node_ – указатель на корень поддерева; key – вставляемый ключ; value – вставляемое значение; opCount – счётчик операций.

Сложность: $O(\log(n))$ – так как происходит спуск по высоте сбалансированного дерева, где n – количество узлов в поддереве.

Дополнительная память: $O(\log(n))$ – из-за глубины рекурсии, где n – количество узлов в поддереве.

➤ **Node*** findNode(**Node*** node_, **const TKey&** key, **size_t&** opCount)

Функционал: Приватный метод. Рекурсивно выполняет поиск узла по ключу в поддереве.

Возвращаемое значение: Node*, указатель на найденный узел или nullptr.

Параметры: node_ – указатель на корень поддерева; key – ключ искомого узла; opCount – счётчик операций.

Сложность: $O(\log(n))$ – так как поиск происходит по пути от корня к листу, где n – количество узлов в поддереве.

Дополнительная память: $O(\log(n))$ – из-за глубины рекурсии, где n – количество узлов в поддереве.

➤ **Node*** maxNode(**Node*** node_, **size_t&** opCount)

Функционал: Приватный метод. Находит узел с максимальным ключом в поддереве (самый правый элемент).

Возвращаемое значение: Node*, указатель на узел.

Параметры: node_ – указатель на корень поддерева; opCount – счётчик операций.

Сложность: $O(\log(n))$ – так как выполняется проход по высоте поддерева, где n – количество узлов в поддереве.

➤ **Node*** removeNode(**Node*** node_, **const TKey&** key, **size_t&** opCount)

Функционал: Приватный метод. Рекурсивно удаляет узел по ключу с последующей балансировкой поддерева.

Возвращаемое значение: Node*, новый корень поддерева.

Параметры: node_ – указатель на корень поддерева; key – ключ удаляемого узла; opCount – счётчик операций.

Сложность: $O(\log(n))$ – так как требуется найти узел и выполнить балансировку по пути вверх, где n – количество узлов в поддереве.

Дополнительная память: $O(\log(n))$ – из-за глубины рекурсии, где n – количество узлов в поддереве.

➤ **void** clearTree(**Node*** node_)

Функционал: Приватный метод. Рекурсивно удаляет все узлы поддерева.

Параметры: node_ – указатель на корень поддерева.

Сложность: $O(n)$ – так как посещается каждый узел поддерева, где n – количество узлов в поддереве.

Дополнительная память: $O(\log(n))$ – из-за глубины рекурсии, где n – количество узлов в поддереве.

➤ **AVLTable()** : root(**nullptr**), NodeCount(**0**) {}

Функционал: Конструктор по умолчанию. Создает пустое дерево.

➤ **void clear() override**

Функционал: Очищает дерево с помощью приватного метода clearTree().

Сложность: $O(n)$ – так как вызывается метод clearTree(), имеющий данную сложность (удаляются все узлы). n – количество узлов в дереве (размер таблицы).

Дополнительная память: $O(\log(n))$ – так как вызывается метод clearTree() (из-за глубины рекурсии). n – количество узлов в дереве (размер таблицы).

➤ **~AVLTable()**

Функционал: Деструктор. Освобождает память, вызывая очистку дерева методом clear().

Сложность: $O(n)$ – из-за вызова метода clear(). n – количество узлов в дереве (размер таблицы).

Дополнительная память: $O(\log(n))$ – из-за вызова метода clear(). n – количество узлов в дереве (размер таблицы).

➤ **std::string getTableName() const override**

Функционал: Возвращает название таблицы.

Возвращаемое значение: std::string, строка "AVLTable".

➤ **size_t size() const override**

Функционал: Возвращает количество элементов в таблице.

Возвращаемое значение: size_t, количество элементов в таблице (количество узлов в дереве).

➤ **size_t insert(const TKey& key, const TValue& value) override**

Функционал: Метод для вставки записи в таблицу. Вставляет элемент в дерево с использованием рекурсивной функции insertNode().

Возвращаемое значение: size_t, количество выполненных операций.

Параметры: key – ключ записи; value – значение записи.

Сложность: $O(\log(n))$ – так как вызывается метод insertNode(). n – количество узлов в дереве (размер таблицы).

Дополнительная память: $O(\log(n))$ – так как вызывается метод insertNode(). n – количество узлов в дереве (размер таблицы).

➤ **FindResult find(const TKey& key) override**

Функционал: Выполняет поиск элемента по ключу. Выполняет поиск элемента в дереве с использованием рекурсивной функции findNode(), рекурсивно сравнивая ключ и переходя в левое или правое поддерево.

Возвращаемое значение: FindResult, указатель на найденное значение и количество выполненных операций.

Параметры: key – ключ поиска.

Сложность: $O(\log(n))$ – так как вызывается метод findNode(). n – количество узлов в дереве (размер таблицы).

Дополнительная память: $O(\log(n))$ – так как вызывается метод findNode(). n – количество узлов в дереве (размер таблицы).

➤ **size_t remove(const TKey& key) override**

Функционал: Метод для удаления записи из таблицы. Удаляет элемент из дерева с использованием рекурсивной функции removeNode().

Возвращаемое значение: size_t, количество выполненных операций.

Параметры: key – ключ удаляемого элемента.

Сложность: $O(\log(n))$ – так как вызывается метод removeNode(). n – количество узлов в дереве (размер таблицы).

Дополнительная память: $O(\log(n))$ – так как вызывается метод removeNode(). n – количество узлов в дереве (размер таблицы).

5) HashTable< TKey, TValue>:

Класс хэш-таблицы, реализованный с использованием метода цепочек для разрешения коллизий. Таблица представляет собой массив списков, в которых хранятся элементы с одинаковыми хэш-значениями. При превышении коэффициента загрузки выполняется рехэширование (увеличение размера таблицы и перераспределение элементов).

Внутренняя структура NodeHash:

- Поле **TKey** key – ключ элемента.
- Поле **TValue** value – значение элемента.
- Поле **NodeHash*** next – указатель на следующий элемент в цепочке.
- Метод NodeHash(**const TKey&** key_, **const TValue&** value_) : key(key_), value(value_), next(nullptr) {}

Функционал: Конструктор узла цепочки. Создаёт элемент с заданным ключом и значением.

Параметры: key_ – ключ; value_ – значение.

Поля класса:

- **std::vector<NodeHash*>** table_lists – массив указателей на головы (первый элемент) цепочек.
- **size_t** sizeTable – текущий размер таблицы (количество цепочек).
- **size_t** elementsCount – количество элементов в таблице.
- **double** loadFactor – коэффициент загрузки, при превышении которого выполняется рехэширование.

Методы класса:

- **size_t hashFunction(const TKey& key_, size_t sizeTable_, size_t& opCount) const**

Функционал: Приватный метод. Вычисляет индекс цепочки в таблице по ключу (полиномиальная хэш-функция).

Возвращаемое значение: size_t, индекс цепочки.

Параметры: key_ – ключ; sizeTable_ – размер таблицы; opCount – счётчик операций.

Сложность: $O(k)$ – так как происходит проход по всем символам в цикле for, где k является длиной ключа (например, строки).

- **NodeHash* findNode(const TKey& key, size_t& opCount) const**

Функционал: Приватный метод. Находит узел по ключу в соответствующей цепочке (после вычисления хэша).

Возвращаемое значение: NodeHash*, указатель на найденный узел или nullptr.

Параметры: key – ключ поиска; opCount – счётчик операций.

Сложность: $O(n)$ – в худшем случае, если все элементы попали в одну цепочку (n – количество элементов в таблице). В среднем $O(1)$, при равномерном распределении по цепочкам.

➤ **void rehash(size_t& opCount)**

Функционал: Приватный метод. Увеличивает размер таблицы (в 2 раза) и перераспределяет все элементы по новым цепочкам с пересчётом хэш-функции.

Параметры: opCount – счётчик операций.

Сложность: $O(n)$ – так как требуется пройти по всем элементам и заново их вставить в новый массив цепочек (n – количество элементов в таблице).

Дополнительная память: $O(n)$ – так как создаётся новый массив цепочек и перераспределяются все элементы таблицы (n – количество элементов в таблице).

➤ **explicit HashTable(size_t sizeTable_ = 10, double loadFactor_ = 0.75)**

Функционал: Конструктор. Инициализирует таблицу заданного размера и коэффициент загрузки.

Параметры: sizeTable_ – начальный размер таблицы; loadFactor_ – коэффициент загрузки.

Сложность: $O(m)$ – так как происходит инициализация вектора размером равным размеру таблицы (m).

Дополнительная память: $O(m)$ – так как выделяется память под цепочки (m штук) в массиве (векторе).

➤ **void clear() override**

Функционал: Удаляет все элементы таблицы, очищая все цепочки.

Сложность: $O(n)$ – так как необходимо пройти по всем элементам таблицы, удалив их (n – количество элементов в таблице).

➤ **~HashTable()**

Функционал: Деструктор. Освобождает память всех узлов и таблицы.

Сложность: $O(n)$ – так как вызывается метод clear(). n – количество элементов в таблице.

➤ **std::string getTableName() const override**

Функционал: Возвращает название таблицы.

Возвращаемое значение: std::string, строка "HashTable".

➤ **size_t size() const override**

Функционал: Возвращает количество элементов в таблице.

Возвращаемое значение: size_t, количество элементов в таблице (количество записей в таблице).

➤ **size_t size_capacity() const**

Функционал: Возвращает текущий размер таблицы (количество цепочек).

Возвращаемое значение: size_t, количество цепочек.

➤ **size_t insert(const TKey& key, const TValue& value) override**

Функционал: Метод для вставки записи в таблицу. Вычисляет хэш и ищет ключ в цепочке: если найден – обновляет значение; если нет – добавляет новый узел в начало цепочки. После вставки проверяется коэффициент загрузки, при необходимости вызывается `rehash()`.

Возвращаемое значение: `size_t`, количество выполненных операций.

Параметры: `key` – ключ записи; `value` – значение записи.

Сложность: $O(1)$ – в среднем; $O(n)$ – в худшем случае при длинной цепочке или во время рехэширования. n – количество элементов в таблице.

Дополнительная память: $O(1)$ или $O(n)$ при рехэшировании. n – количество элементов в таблице.

➤ **FindResult find(const TKey& key) override**

Функционал: Выполняет поиск элемента в цепочке по вычисленному из ключа хэшу.

Возвращаемое значение: FindResult, указатель на найденное значение и количество выполненных операций.

Параметры: `key` – ключ поиска.

Сложность: $O(1)$ – в среднем; $O(n)$ – в худшем случае при длинной цепочке. n – количество элементов в таблице.

➤ **size_t remove(const TKey& key) override**

Функционал: Метод для удаления записи из таблицы. Находит элемент в цепочке и удаляет его, с учётом случая удаления первого элемента цепочки или внутреннего элемента.

Возвращаемое значение: `size_t`, количество выполненных операций.

Параметры: `key` – ключ удаляемого элемента.

Сложность: $O(1)$ – в среднем; $O(n)$ – в худшем случае при длинной цепочке. n – количество элементов в таблице.

6) LoggingOperations:

Класс для логирования операций, выполняемых над таблицами. Сохраняет информацию о каждой операции (тип таблицы, тип операции, количество операций), а также накапливает суммарную статистику по операциям и по типам таблиц.

Внутренняя структура InputLogs:

- Поле **std::string** `tableName` – название таблицы.
- Поле **std::string** `operationName` – название операции (insert, find, remove).
- Поле **size_t** `operationsCount` – количество операций.

Поля класса:

- **std::vector<InputLogs>** `logs` – динамический массив логов операций.
- **size_t** `totalInsert` – суммарное количество операций вставки.
- **size_t** `totalFind` – суммарное количество операций поиска.
- **size_t** `totalRemove` – суммарное количество операций удаления.
- **size_t** `totalUnorderedOp` – суммарное количество операций для UnorderedTable.
- **size_t** `totalAVLOp` – суммарное количество операций для AVLTable.
- **size_t** `totalHashOp` – суммарное количество операций для HashTable.

Методы класса:

➤ **LoggingOperations()**

Функционал: Конструктор по умолчанию. Инициализирует все счётчики нулевыми значениями.

➤ **void add(const std::string& tableName_, const std::string& operationName_, size_t operationsCount)**

Функционал: Добавляет запись о выполненной операции в лог и обновляет суммарные счётчики по операциям и типам таблиц.

Параметры: tableName_ – название таблицы; operationName_ – тип операции; operationsCount – количество операций.

➤ **void writeToFile(const std::string& fileName) const**

Функционал: Записывает все логи и суммарную статистику в файл.

Параметры: fileName – имя файла.

Сложность: $O(n)$ – так как выполняется проход по всем логам, где n – количество записей в logs.

➤ **void print() const**

Функционал: Выводит все логи в консоль.

Сложность: $O(n)$ – так как выполняется проход по всем логам, где n – количество записей в logs.

➤ **void clear()**

Функционал: Очищает все логи и обнуляет суммарные счётчики.

Сложность: $O(n)$ – так как выполняется очистка вектора logs методом clear(), где n – количество записей в logs.

7) MainTable< TKey, TValue>:

Класс-обёртка, объединяющий несколько реализаций таблиц (UnorderedTable, AVLTable, HashTable). Обеспечивает одновременное выполнение операций над всеми таблицами и логирование результатов.

Поля класса:

- **UnorderedTable< TKey, TValue>** unorderedTable – неупорядоченная таблица.
- **AVLTable< TKey, TValue>** avlTable – таблица на основе АВЛ-дерева.
- **HashTable< TKey, TValue>** hashTable – хэш-таблица.
- **LoggingOperations** logger – объект для логирования операций.

Методы класса:

➤ **size_t insertALL(const TKey& key, const TValue& value)**

Функционал: Выполняет вставку элемента во все таблицы. Для каждой таблицы сохраняется количество операций в лог. Возвращает суммарное количество операций.

Возвращаемое значение: size_t, суммарное количество операций по всем таблицам.

Параметры: key – ключ записи; value – значение записи.

Сложность: $O(n)$ – определяется суммой сложностей вставки во все таблицы ($O(n)$ для UnorderedTable, $O(\log n)$ для AVLTable, $O(1)$ в среднем для HashTable). Доминирует $O(n)$.

Дополнительная память: $O(\log(n))$ – определяется суммой затрат на дополнительную память вставки во все таблицы ($O(1)$ для UnorderedTable, $O(\log n)$ для AVLTable, $O(1)$ в среднем для HashTable). Доминирует $O(\log(n))$.

➤ **FindResult** findALL(const TKey& key)

Функционал: Выполняет поиск элемента во всех таблицах. Логирует количество операций для каждой таблицы. Возвращает суммарное количество операций и указатель на найденное значение (приоритет: Unordered => AVL => Hash).

Возвращаемое значение: FindResult, указатель на найденное значение и суммарное количество выполненных операций по всем таблицам.

Параметры: key – ключ поиска.

Сложность: $O(n)$ – определяется суммой сложностей поиска во всех таблицах ($O(n)$ для UnorderedTable, $O(\log n)$ для AVLTable, $O(1)$ в среднем для HashTable). Доминирует $O(n)$.

Дополнительная память: $O(\log(n))$ – определяется суммой затрат на дополнительную память поиска во всех таблицах ($O(1)$ для UnorderedTable, $O(\log n)$ для AVLTable, $O(1)$ в среднем для HashTable). Доминирует $O(\log(n))$.

➤ **size_t** removeALL(const TKey& key)

Функционал: Выполняет удаление элемента из всех таблиц. Логирует количество операций. Возвращает суммарное количество операций.

Возвращаемое значение: size_t, суммарное количество операций по всем таблицам.

Параметры: key – ключ удаляемого элемента.

Сложность: $O(n)$ – определяется суммой сложностей удаления во всех таблицах ($O(n)$ для UnorderedTable, $O(\log n)$ для AVLTable, $O(1)$ в среднем для HashTable). Доминирует $O(n)$.

Дополнительная память: $O(\log(n))$ – определяется суммой затрат на дополнительную память удаления во всех таблицах ($O(1)$ для UnorderedTable, $O(\log n)$ для AVLTable, $O(1)$ в среднем для HashTable). Доминирует $O(\log(n))$.

➤ **bool** is_emptyALL() const

Функционал: Проверяет, пусты ли все таблицы.

Возвращаемое значение: bool, true, если все таблицы пусты; false, иначе.

➤ **void** saveLogs(const std::string& fileName)

Функционал: Сохраняет логи операций в файл через объект logger.

Параметры: fileName – имя файла.

Сложность: $O(n)$ – так как выполняется проход по всем логам, где n – количество записей в logs (вызов метода writeToFile()).

3.3. Описание алгоритмов

Алгоритмы, связанные с *Polynom.h* были подробно описаны в отчёте для лабораторной работы №1 «Арифметические операции с полиномами».

1) Алгоритм вставки в неупорядоченную таблицу (**UnorderedTable::insert**).

Вставка элемента в неупорядоченную таблицу на основе вектора выполняется линейным поиском:

- Последовательно просматриваются все записи вектора.
- Если запись с заданным ключом обнаружена, её значение обновляется, и метод завершается.
- Если ключ не найден, новая запись добавляется в конец вектора методом **push_back()**.
- Количество выполненных операций (сравнений, присваиваний и т.д.) фиксируется счётчиком **opCount**.

Алгоритм работает за **O(n)** в худшем и среднем случаях, где n – текущий размер таблицы.

2) Алгоритм поиска в неупорядоченной таблице (**UnorderedTable::find**).

Поиск осуществляется последовательным просмотром всех элементов вектора:

- Для каждой записи проверяется равенство ключа.
- При совпадении формируется результат **FindResult**, содержащий указатель на найденное значение и количество выполненных операций.
- Если ключ не найден, возвращается **FindResult** с нулевым указателем.

Алгоритм работает за **O(n)**, где n – текущий размер таблицы.

3) Алгоритм удаления из неупорядоченной таблицы (**UnorderedTable::remove**).

Удаление состоит из двух этапов – поиска и собственно удаления:

- Производится линейный поиск записи с заданным ключом.
- При обнаружении элемента он заменяется последним элементом вектора, после чего вызывается **pop_back()** для уменьшения размера.
- Если ключ не найден, таблица остаётся без изменений.

Алгоритм работает за **O(n)** (доминирует поиск), где n – текущий размер таблицы.

4) Алгоритм вставки в АВЛ-дерево (**AVLTable::insert**).

Вставка элемента в упорядоченную таблицу на базе АВЛ-дерева реализована рекурсивно:

- Спуск по дереву происходит с помощью лексикографического сравнения ключей (**operator<**, **operator>**).
- Если узел с таким ключом уже существует, его значение обновляется.
- Иначе создаётся новый лист, после чего на обратном пути из рекурсии для каждого пройденного узла вызывается балансировка (**balancingTree()**).
- Балансировка вычисляет фактор баланса (разность высот правого и левого поддеревьев). Если модуль фактора баланса больше или равен 2, применяется один из четырёх поворотов:

- малый левый (**smallLeftRotation()**) или малый правый (**smallRightRotation()**);
- большой левый (**bigLeftRotation()**) или большой правый (**bigRightRotation()**) – в случаях, когда требуется двойной поворот.

- После поворотов высоты узлов пересчитываются.

Алгоритм работает за **$O(\log(n))$** по времени и требует **$O(\log(n))$** дополнительной памяти (рекурсия), где n – количество узлов в дереве.

5) Алгоритм поиска в АВЛ-дереве (**AVLTable::find**).

Поиск элемента выполняется рекурсивным обходом:

- На каждом шаге ключ сравнивается лексикографически с ключом текущего узла.
- При равенстве ключей узел найден, и результат возвращается.
- Иначе поиск продолжается в левом или правом поддереве в зависимости от результата сравнения.
- При совпадении ключей (если **findNode()** вернул не nullptr) формируется результат **FindResult**, содержащий указатель на найденное значение и количество выполненных операций.
- Если ключ не найден, возвращается **FindResult** с нулевым указателем.

Алгоритм работает за **$O(\log(n))$** , где n – количество узлов в дереве.

6) Алгоритм удаления из АВЛ-деревя (**AVLTable::remove**).

Удаление выполняется рекурсивно и включает поиск узла с последующей балансировкой:

- Если узел не найден, дерево не изменяется.
- При нахождении удаляемого узла возможны три случая:
 - Узел не имеет потомков – просто удаляется.
 - Узел имеет одного потомка – заменяется этим потомком.
 - Узел имеет двух потомков – находится максимальный узел в левом поддереве (**maxNode()**), его ключ и значение копируются в удаляемый узел, после чего рекурсивно удаляется сам максимальный элемент.
- После удаления выполняется балансировка всех узлов, затронутых операцией.

Алгоритм работает за **$O(\log(n))$** , где n – количество узлов в дереве.

7) Хэш-функция (**HashTable::hashFunction**).

Используется полиномиальное хэширование строкового ключа:

- Хэш вычисляется по формуле **$hashValue = hashValue * 31 + (\text{unsigned char})key_ [i]$** для каждого символа строки.
- Итоговое значение берётся по остатку от деления на текущий размер таблицы **$hashValue \% sizeTable_$** .
- Коэффициент 31 и целочисленное умножение обеспечивают равномерное распределение.

Алгоритм работает за **$O(k)$** , где k – длина строки-ключа.

8) Алгоритм вставки в хэш-таблицу с цепочками (**HashTable::insert**).

Вставка элемента в хэш-таблицу, реализованную методом цепочек, состоит из следующих шагов:

- По ключу вычисляется индекс цепочки с помощью хэш-функции.
- В односвязном списке (этой цепочке) выполняется поиск ключа. Если ключ найден, значение обновляется.
- Если ключ не найден, создаётся новый узел, который вставляется в начало цепочки.
- После вставки проверяется коэффициент загрузки **elementsCount > loadFactor * sizeTable**. Если он превышен, вызывается функция рехэширования (**rehash()**), увеличивающая размер таблицы вдвое и перераспределяющая все элементы.

Алгоритм работает за **O(1)** в среднем и за **O(n)** в худшем случаях (при длинных цепочках или при рехэшировании), где n – текущий размер таблицы (количество записей).

9) Алгоритм поиска в хэш-таблице (**HashTable::find**).

Поиск элемента выполняется следующим образом:

- Вычисляется индекс цепочки по ключу.
- В соответствующей цепочке производится линейный просмотр узлов до обнаружения искомого ключа или достижения конца цепочки (**findNode()**).
- При совпадении ключей (если **findNode()** вернул не nullptr) формируется результат **FindResult**, содержащий указатель на найденное значение и количество выполненных операций.
- Если ключ не найден, возвращается **FindResult** с нулевым указателем.

Алгоритм работает за **O(1)** в среднем и за **O(n)** в худшем случаях, где n – текущий размер таблицы (количество записей).

10) Алгоритм удаления из хэш-таблицы (**HashTable::remove**).

Удаление элемента аналогично поиску:

- Определяется индекс корзины через хэш-функцию.
- В цепочке ищется узел с заданным ключом; предыдущий узел запоминается; при нахождении удаляемый элемент исключается из цепочки; корректируется указатель, связывая предыдущий узел со следующим узлом после удаляемого узла.
- Память узла освобождается, счётчик элементов уменьшается.

Алгоритм работает за **O(1)** в среднем и за **O(n)** в худшем случаях, где n – текущий размер таблицы (количество записей).

11) Алгоритм рехэширования (**HashTable::rehash**).

Рехэширование запускается при превышении коэффициента загрузки:

- Создаётся новый вектор цепочек удвоенного размера.
- Каждый элемент текущей таблицы извлекается, для него заново вычисляется хэш относительно нового размера, после чего элемент вставляется в начало соответствующей цепочки нового вектора.
- Старая таблица замещается новой, а размер таблицы обновляется.

Алгоритм работает за **O(n)** по времени и требует **O(n)** дополнительной памяти, где n – количество элементов в таблице.

12) Подсчёт количества элементарных операций.

Для сбора статистики, используемой в экспериментах и логировании, во всех методах таблиц (**insert()**, **find()**, **remove()**) присутствует локальный счётчик **size_t opCount**.

- Счётчик увеличивается на единицу при каждом сравнении, присваивании, арифметическом действии и других логически значимых шагах.
- Итоговое значение возвращается (для **insert()/remove()**) или сохраняется в поле **operationsCount** структуры **FindResult** для **find()**.
- Это позволяет для каждого типа таблицы зафиксировать точное количество элементарных операций, затраченных на выполнение действия, и объективно сравнивать эффективность структур.

4. Результаты экспериментов и тестирования

Для проверки корректности работы программы проведено комплексное модульное тестирование с использованием **Google Test**. Тесты охватывают ключевые функции всех классов таблиц (**UnorderedTable**, **AVLTable**, **HashTable**), класс **MainTable**, а также взаимодействие таблиц с полиномами. В **экспериментальной части** выполнено сравнение эффективности операций вставки, поиска и удаления при различных объёмах данных.

Тестовое покрытие включает следующие логические группы:

1) Тестирование класса **UnorderedTable**.

- Создание таблицы::

Тест **CreateIsCorrect** подтвердил, что сразу после создания неупорядоченной таблицы метод `size()` возвращает 0, а `getTableName()` – строку "UnorderedTable". Таким образом, таблица инициализируется в полностью пустом состоянии.

- Вставка и поиск:

Тест **InsertAndFind** проверил базовый сценарий: в таблицу последовательно вставлены записи с ключами "a" (значение 10) и "b" (значение 20). Последующий поиск через `find()` для обоих ключей вернул непустые указатели, а разыменованные значения в точности совпали с вставленными (10 и 20). Это подтверждает корректность вставки и поиска.

- Перезапись существующего ключа:

Тест **OverwriteKey** продемонстрировал, что при повторной вставке с уже существующим ключом "a" (сначала 10, затем 50) значение обновляется: `find("a")` возвращает 50. Размер таблицы при этом остаётся равным 1, что говорит о том, что новая запись не дублируется, а замещает старую.

- Поиск отсутствующего ключа:

Тест **FindNonExistent** показал, что при поиске ключа "x", которого нет в таблице, метод `find()` возвращает `FindResult` с `value == nullptr`. Это гарантирует корректное распознавание отсутствующих элементов.

- Удаление существующего элемента:

Тест **Remove** проверил сценарий: после вставки двух записей ("a" и "b") удаляется запись с ключом "a". Повторный поиск "a" возвращает нулевой указатель, а поиск "b" выдаёт значение 20. Размер таблицы после удаления корректно уменьшен с 2 до 1.

- Удаление несуществующего элемента:

Тест **RemoveNonExistent** подтвердил, что попытка удалить ключ "x" после вставки только одного элемента "a" не изменяет размер таблицы (остаётся 1). Никаких исключений не выбрасывается.

- Очистка таблицы:

Тест **Clear_GetTableName_Size** проверил, что после вставки двух элементов метод `clear()` полностью очищает таблицу: последующий

вызов `size()` возвращает 0, а `getTableName()` корректно возвращает имя таблицы "UnorderedTable".

2) Тестирование класса **AVLTable**.

- Создание таблицы:

Тест **CreateIsCorrect** удостоверился, что новая АВЛ-таблица создаётся пустой (`size() == 0`) и имеет имя "AVLTable".

- Вставка и поиск:

Тест **InsertAndFind** выполнил последовательную вставку трёх записей с ключами "a", "b", "c" и значениями 1, 2, 3. Для каждого ключа поиск вернул указатель на ожидаемое значение. Это подтверждает корректность вставки и поиска в сбалансированном дереве.

- Перезапись существующего ключа:

Тест **OverwriteKey** показал, что вставка с ключом "a" сначала со значением 1, затем с 99, приводит к обновлению значения (поиск возвращает 99). Число узлов дерева (`size()`) сохраняется равным 1.

- Поиск отсутствующего ключа:

Тест **FindNonExistent** подтвердил, что поиск ключа "x" в таблице, где такого элемента нет, возвращает нулевой указатель.

- Удаление листового узла:

Тест **RemoveLeaf** проверил удаление элемента, не имеющего потомков. В дерево добавлены ключи "b" (корень) и "a" (левый лист). После удаления "a" этот ключ больше не находится, размер дерева становится равным 1, а ключ "b" остаётся доступным. Структура дерева сохраняет корректность.

- Удаление узла с одним потомком:

Тест **RemoveNodeWithOneChild** протестировал случай, когда удаляемый узел имеет ровно одного потомка. Дерево построено из ключей "c", "b", "a" в порядке убывания. Удаляется "b". После этого "b" не найден, а элементы "a" и "c" по-прежнему доступны. Балансировка сохраняет корректность дерева.

- Удаление узла с двумя потомками:

Тест **RemoveNodeWithTwoChild** проверил удаление корневого узла "b", у которого есть оба потомка ("a" слева и "c" справа). После удаления "b" ключ не найден, однако записи "a" и "c" остаются и возвращают правильные значения. Это подтверждает корректную замену удаляемого узла максимальным элементом левого поддерева.

- Удаление несуществующего элемента:

Тест **RemoveNonExistent** показал, что попытка удаления ключа "x" после вставки "a" не влияет на дерево: `size()` остаётся равным 1.

- Очистка таблицы:

Тест **Clear_GetTableName_Size** подтвердил, что после вставки двух элементов вызов `clear()` обнуляет счётчик узлов и `getTableName()` корректно возвращает имя таблицы "AVLTable".

3) Тестирование класса **HashTable**.

- Создание таблицы:

Тест **CreateIsCorrect** проверил, что хэш-таблица, созданная по умолчанию, имеет размер 0 и имя "HashTable".

- Вставка и поиск:

Тест **InsertAndFind** вставил записи "a":10 и "b":20, после чего поиск обоих ключей вернул ненулевые указатели с ожидаемыми значениями.

- Перезапись существующего ключа:

Тест **OverwriteKey** продемонстрировал, что повторная вставка ключа "a" с новым значением 50 обновляет запись (поиск даёт 50), при этом общее количество элементов elementsCount остаётся равным 1.

- Поиск отсутствующего ключа:

Тест **FindNonExistent** удостоверился, что поиск ключа "x", не добавленного в таблицу, возвращает нулевой указатель.

- Удаление существующего элемента:

Тест **Remove** проверил: после вставки "a" и "b" удаление "a" приводит к тому, что "a" больше не находится, тогда как "b" по-прежнему доступен. Размер таблицы уменьшается до 1.

- Удаление несуществующего элемента:

Тест **RemoveNonExistent** показал, что попытка удалить несуществующий ключ "x" после вставки элемента "a" не меняет количество записей (остаётся 1).

- Проверка рехэширования:

Тест **RehashTrigger** создал таблицу с начальным размером 2 и коэффициентом загрузки 0.5. После вставки двух элементов ("a", "b") произошло автоматическое рехэширование: ёмкость таблицы (size_capacity()) увеличилась до 4, а оба элемента остались доступными и возвращают правильные значения (1 и 2). Это подтверждает работоспособность механизма расширения.

- Очистка таблицы:

Тест **Clear_GetTableName_Size** проверил, что после вставки двух записей метод clear() полностью очищает таблицу (размер 0), а getTableName() корректно возвращает имя таблицы "HashTable".

4) Тестирование класса MainTable.

- Вставка и поиск через общий интерфейс:

Тест **InsertALLAndFindALL** выполнил вставку записей "a":10 и "b":20 одновременно во все три таблицы с помощью insertALL(). Последующий вызов findALL() для каждого ключа вернул непустой указатель и правильные значения (10 и 20). Результат подтвердил синхронную работу всех таблиц и корректное объединение результатов поиска (приоритет от UnorderedTable к HashTable).

- Удаление через общий интерфейс:

Тест **RemoveALL** добавил две записи, затем удалил "a" через removeALL(). Поиск "a" после этого вернул нулевой указатель, а поиск "b" – значение 20. Это доказывает корректное удаление из всех трёх таблиц.

- Проверка пустоты всех таблиц:

Тест **is_emptyALL** подтвердил, что для нового созданного объекта класса **MainTable** метод **is_emptyALL()** возвращает **true**, а после вставки одного элемента – **false**.

5) Тестирование **Polynom_Table_Tests**.

- Неупорядоченная таблица с полиномами:

Тест **UnorderedTableWithPolynomial** создал полином из одного монома ($1.0 \cdot x^1 y^1 z^1$, степень 111), вставил его в неупорядоченную таблицу с ключом "a", а затем проверил: полином не пуст, его коэффициент равен 1.0, степень – 111. Это подтверждает корректную работу таблицы с пользовательским типом данных **Polynomial**.

- АВЛ-таблица с полиномами:

Тест **AVLTableWithPolynomial** выполнил аналогичную проверку для АВЛ-таблицы: полином ($2.0 \cdot x^2 y^2 z^2$, степень 222) был вставлен и успешно извлечён, причём значения коэффициента и степени совпали с исходными.

- Хэш-таблица с полиномами:

Тест **HashTableWithPolynomial** проверил сохранение полинома ($3.0 \cdot x^3 y^3 z^3$, степень 333) в хэш-таблице. Поиск вернул полином с правильным коэффициентом и степенью, что свидетельствует о полной совместимости хэш-таблицы с полиномиальным типом.

Эксперименты включают следующие логические группы:

Цель экспериментов – сравнить эффективность операций вставки, поиска и удаления в трёх типах таблиц при увеличении объёма данных. Для каждого эксперимента генерировались наборы из **N** полиномов (каждый полином является простым мономом) с уникальными строковыми ключами ("key0", "key1", ...). Измерялось суммарное количество элементарных операций (**opCount**) и время выполнения (в секундах) для каждой таблицы. Эксперименты проведены для **N** = 1000, 5000, 10000, 20000.

1) Эксперимент **ExperimentInsert** (сравнение вставки).

- Каждая таблица заполнялась с нуля последовательной вставкой всех **N** элементов. Результаты:

```
+-----+INSERT_EXPERIMENT+-----+
N = 1000:
Unordered: opCount = 2003000, Time = 0.0152936 seconds
AVL: opCount = 415388, Time = 0.002112 seconds
Hash: opCount = 158397, Time = 0.0011228 seconds

N = 5000:
Unordered: opCount = 50015000, Time = 0.276705 seconds
AVL: opCount = 2647065, Time = 0.0107629 seconds
Hash: opCount = 751901, Time = 0.0060884 seconds

N = 10000:
Unordered: opCount = 200030000, Time = 1.16634 seconds
AVL: opCount = 5746784, Time = 0.0246595 seconds
Hash: opCount = 1539833, Time = 0.0115294 seconds

N = 20000:
Unordered: opCount = 800060000, Time = 4.01711 seconds
AVL: opCount = 12503850, Time = 0.0505016 seconds
Hash: opCount = 3190183, Time = 0.0239439 seconds
+-----+-----+
```

Рис. 8. Вставка элементов в таблицы трёх типов

- **Анализ:** Вставка в неупорядоченную таблицу показывает квадратичный рост числа операций и времени, что объясняется линейным поиском дубликатов при каждой вставке. АВЛ-дерево демонстрирует логарифмическую зависимость, а хэш-таблица с методом цепочек – почти постоянное среднее время, благодаря эффективному распределению ключей и редким рехэшированиям.

2) Эксперимент ExperimentFind (сравнение поиска).

- После заполнения всех таблиц N элементами выполнялся поиск каждого из N ключей. Результаты:

```
+-----+FIND_EXPERIMENT+-----+
N = 1000:
Unordered: opCount = 2008000, Time = 0.0109138 seconds
AVL: opCount = 31708, Time = 0.0005265 seconds
Hash: opCount = 50412, Time = 0.0001442 seconds

N = 5000:
Unordered: opCount = 50040000, Time = 0.255693 seconds
AVL: opCount = 195224, Time = 0.0028278 seconds
Hash: opCount = 289008, Time = 0.0010344 seconds

N = 10000:
Unordered: opCount = 200080000, Time = 1.06462 seconds
AVL: opCount = 419290, Time = 0.0062332 seconds
Hash: opCount = 578310, Time = 0.0018626 seconds

N = 20000:
Unordered: opCount = 800160000, Time = 3.80589 seconds
AVL: opCount = 905642, Time = 0.0132878 seconds
Hash: opCount = 1206504, Time = 0.0052246 seconds
+-----+-----+
```

Рис. 9. Поиск элементов в таблицах трёх типов

- **Анализ:** Для поиска неупорядоченная таблица вновь демонстрирует квадратичную сложность при последовательных запросах, так как каждый поиск требует линейного перебора. АВЛ-дерево и хэш-таблица выполняют поиск значительно быстрее; хэш-таблица сохраняет лидерство благодаря почти мгновенному вычислению индекса.

3) Эксперимент ExperimentRemove (сравнение удаления).

- После заполнения таблиц N элементами выполнялось последовательное удаление всех N ключей. Результаты:

```

+-----+REMOVE_EXPERIMENT+-----+
N = 1000:
Unordered: opCount = 1007000, Time = 0.0066151 seconds
AVL: opCount = 290482, Time = 0.0015446 seconds
Hash: opCount = 51361, Time = 0.001018 seconds

N = 5000:
Unordered: opCount = 25035000, Time = 0.145141 seconds
AVL: opCount = 1903969, Time = 0.0096274 seconds
Hash: opCount = 288826, Time = 0.0045104 seconds

N = 10000:
Unordered: opCount = 100070000, Time = 0.581055 seconds
AVL: opCount = 4084135, Time = 0.0204276 seconds
Hash: opCount = 580845, Time = 0.0091505 seconds

N = 20000:
Unordered: opCount = 400140000, Time = 2.03084 seconds
AVL: opCount = 8961473, Time = 0.0417663 seconds
Hash: opCount = 1221571, Time = 0.0182822 seconds
+-----+

```

Рис. 10. Удаление элементов из таблиц трёх типов

- **Анализ:** Удаление из неупорядоченной таблицы также имеет квадратичную сложность из-за необходимости поиска элемента перед удалением. AVL-дерево и хэш-таблица и здесь демонстрируют существенно лучшие показатели; хэш-таблица оказывается наиболее быстрой структурой для всех операций.

Анализ результатов:

Все модульные тесты успешно пройдены – как для базовых типов данных, так и для полиномов. Подтверждена корректность операций вставки (включая обновление существующих ключей), поиска (в том числе отсутствующих элементов), удаления (листовых узлов, узлов с одним и двумя потомками в AVL-дереве), а также механизма рехэширования в хэш-таблице. Класс MainTable корректно синхронизирует действия над всеми таблицами и объединяет результаты.

Результаты экспериментов полностью соответствуют теоретическим ожиданиям:

- **Неупорядоченная таблица** на векторе показывает квадратичный рост суммарных операций при последовательной обработке N элементов ($O(n)$ на одну операцию, $O(n^2)$ для N операций), что делает её неэффективной для больших объёмов.
- **AVL-таблица** демонстрирует логарифмическую сложность операций ($O(\log n)$ на один запрос), благодаря автоматической балансировке, и остаётся стабильной при увеличении объёма данных.
- **Хэш-таблица** с методом цепочек обеспечивает в среднем константное время ($O(1)$), и её преимущество особенно заметно на больших наборах.

Проведённое тестирование и эксперименты подтверждают корректность функционирования всех разработанных структур данных и их соответствие требованиям задачи, а также предоставляют объективные данные для сравнения их производительности.

Заключение

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

- 1. Реализация таблиц и управляющего класса:** Разработаны классы `UnorderedTable` (на основе вектора), `AVLTable` (на сбалансированном АВЛ-дереве) и `HashTable` (с методом цепочек), поддерживающие вставку, поиск и удаление записей с подсчётом элементарных операций. Создан класс `MainTable`, синхронно выполняющий операции над всеми таблицами, и система логирования, сохраняющая накопленную статистику в файл.
- 2. Консольное приложение и обработка ошибок:** Реализован пользовательский интерфейс, позволяющий вводить полиномы, производить над ними арифметические операции и размещать результаты в таблицах, а также работать с целыми, вещественными, строковыми, булевыми и символьными данными. Программа корректно обрабатывает некорректный ввод и исключительные ситуации без аварийного завершения.
- 3. Тестирование и эксперименты:** С помощью `Google Test` проведено модульное тестирование всех классов; проверены основные операции, граничные случаи (обновление значений, удаление узлов с разным числом потомков, рехэширование) и работа с полиномами. Эксперименты на наборах от 1000 до 20000 элементов показали квадратичный рост затрат для неупорядоченной таблицы и существенно более высокую производительность АВЛ-дерева и хэш-таблицы, что полностью соответствует теоретическим оценкам сложности.

Таким образом, разработанная программа удовлетворяет всем требованиям задания, обеспечивает корректное функционирование таблиц трёх типов и подтверждена тестированием и экспериментальным анализом.

Литература

1. Сайт cppreference: <https://en.cppreference.com/>